

Diseño lógico de un Ensamblador de Tramas de Telemetría basado en FSM

Paul A. Landeo A., Ana L. I. Díaz Y., Gerardo A. Moreno M., Joseph D. Terrones L.

*E.P. Ingeniería de Telecomunicaciones, Universidad Nacional Mayor de San Marcos
Lima, Perú*

Diseño de Arquitectura y Lógica Estructural

Diagrama Esquemático Global:

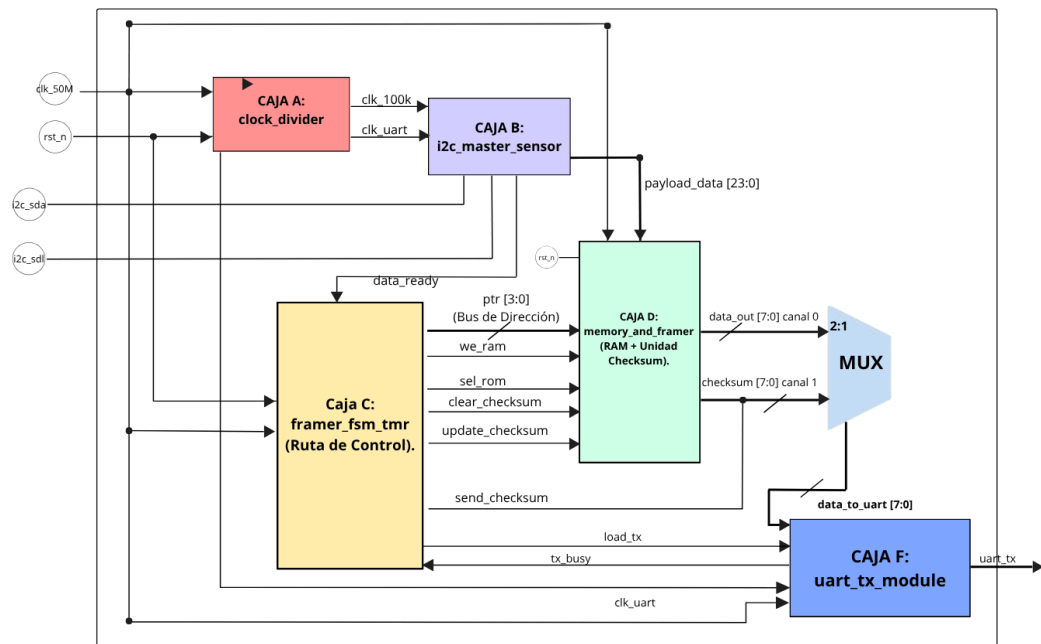


Figura 1. Diagrama Esquemático Global

El núcleo del sistema es un Ensamblador de Tramas de Telemetría (`top_telemetry_framer`), diseñado específicamente para operar en entornos de alta radiación, como misiones satelitales. Su arquitectura se fundamenta en la separación estricta entre las rutas de control y de datos (Datapath). El ciclo operativo comienza cuando el bloque `clock_divider` procesa la señal de reloj maestra (`clk_50M`), generando las sub-frecuencias `clk_100k` y `clk_uart` para sincronizar los periféricos. Posteriormente, el módulo `i2c_master_sensor` recolecta la carga útil de los sensores mediante el bus `i2c_sda` y la línea `i2c_scl`, entregando un bus paralelo de 176 bits y un pulso de sincronismo (`data_ready`) a la unidad de control central.

La gestión del sistema recae en la máquina de estados finitos `framer_fsm_tmr`, la cual emplea Redundancia Triple Modular (TMR) y votación mayoritaria para neutralizar errores inducidos por radiación. Esta FSM supervisa al bloque `memory_and_framer`, regulando la lectura mediante el puntero `ptr[3:0]`, la escritura (`we_ram`) y el cálculo del byte de redundancia (Checksum). En la etapa final, un multiplexor 2:1, bajo la señal `send_checksum`, integra el bus de datos (`data_out`) con el checksum verificado. La trama resultante de 8 bits se transfiere al módulo `uart_tx_module`, encargado de la transmisión serie bit a bit hacia el exterior a través del puerto físico `uart_tx`.

Mapa de Memoria y Registros:

Dado que nuestro puntero de memoria mem_ptr es de 4 bits, tiene un espacio de 16 direcciones disponibles para definir la trama de telemetría. Según la lógica de sel_rom y we_ram, esta es la distribución propuesta:

Tabla 1. Distribución de direcciones de memoria

Dirección (mem_ptr)	Componente	Acceso	Descripción
0x0 - 0x3 (0-3)	Frame Header (ROM)	Solo Lectura	Identificador de misión y palabras de sincronización fija (4 bytes).
0x3 - 0x17 (0-23)	Payload RAM	R/W	Almacena los 192 bits (24 bytes) de datos síncronos extraídos de los sensores (14B MPU6050, 8B BME280, 2B ADC). (El puntero se reinicia a 0 tras la ROM).
0x1A - 0x1B (26-27)	Status Regs	Lectura	Estado operativo de las transiciones I2C y telemetría de salud interna (2 bytes).
0x1C (28)	Checksum Val	Lectura	Registro que almacena el byte de redundancia cíclica.
0x1D - 0x1F (29-31)	Reserved	-	Espacio libre de direccionamiento mapeado para la expansión de sensores adicionales.

Diseño de la Unidad de Control (FSM)

Diagrama FSM

La FSM del sistema de telemetría es una máquina de Mealy de siete estados que gobierna el ensamblador en la FPGA. Con una estructura determinista y tolerante a fallos (TMR), gestiona desde la espera inicial (IDLE) hasta el cierre de trama (DONE), coordinando la captura I2C, el almacenamiento en RAM, la conmutación a ROM para la cabecera, el envío de carga útil y la inyección del checksum.

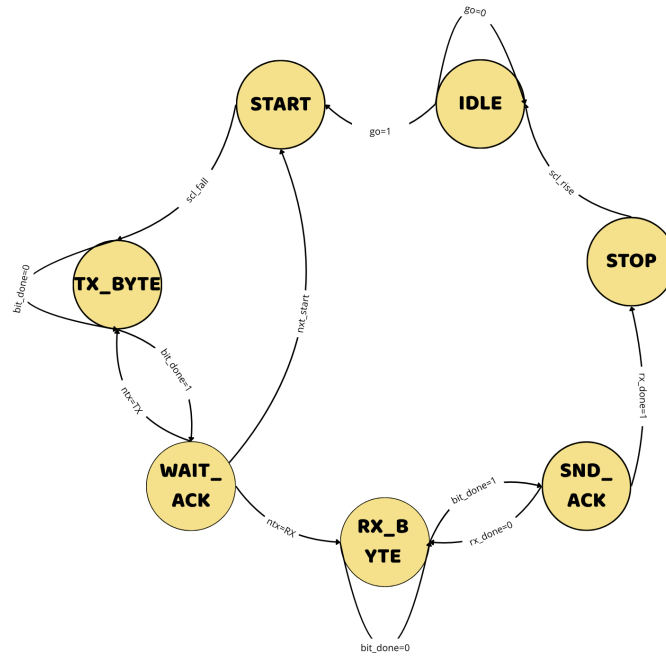


Figura 2. Diagrama FSM (Finite State Machines)

Diagrama de Transición de Estados (ASM)

El Diagrama ASM muestra el algoritmo de control del sistema y el flujo que evidencia un diseño tipo Mealy, donde las señales dependen combinatorialmente del estado actual y de las banderas de estado.

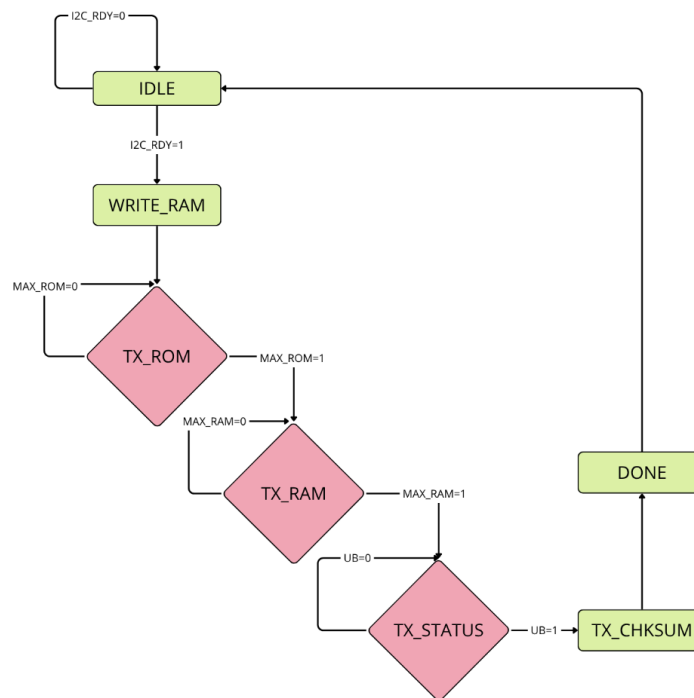


Figura 3. Diagrama ASM (Algorithmic State Machine)

Tabla 2. Tabla de Entradas y Salidas del Diagrama ASM

Entradas (Condiciones Lógicas)	Salidas (Acciones de Control)
data_ready_pulse (I2C_RDY): Pulso de un ciclo de reloj que alerta a la FSM que el módulo maestro finalizó el barrido secuencial (polling) de los 22 bytes de los periféricos.	we_ram: Señal de habilitación de escritura síncrona en los bloques M9K de la RAM para almacenar la ráfaga de datos del sensor.
txb_active (tx_busy): Señal de nivel proveniente de la UART que indica que el módulo de transmisión se encuentra ocupado serializando un byte.	sel_rom: Señal combinacional de selección para el multiplexor de memoria (1: activa el banco ROM de cabecera; 0: activa el banco RAM/Status).
txb_falling: Detector de flanco de bajada que notifica a la FSM el fin exacto de la transmisión de un byte UART para poder avanzar.	load_tx: Pulso síncrono enviado al módulo UART para capturar el dato presente en el bus e iniciar de inmediato la serialización bit a bit.
ptr_reg == 5'd3 (MAX_ROM): Condición lógica interna que se activa cuando el puntero alcanza el límite de lectura de la cabecera fija de la ROM.	ptr [4:0]: Bus de dirección físico de 5 bits que indexa de forma síncrona las posiciones de los bancos de memoria interna.
ptr_reg == 5'd23 (MAX_RAM): Condición de límite que indica que el puntero llegó al final del bloque de datos útil de los sensores en la RAM.	clear_checksum: Señal síncrona de borrado que restablece a 0x00 el registro acumulador de la unidad aritmética XOR.
	update_checksum: Strobe de control síncrono que ordena realizar la operación lógica XOR del byte actual en tránsito.
	send_checksum: Señal de control que conmuta el multiplexor final de salida para inyectar el byte de checksum hacia la UART.

Tablas de Verdad y Transición:

Tabla de Transición de Estados:

Usaremos 3 bits para los 7 estados:

Tabla 3. Tabla de Estados de la FSM

Q2	Q1	Q0	Estados	Descripción Funcional
0	0	0	IDLE	Estado de reposo. Espera el pulso de captura del sensor I2C.
0	0	1	WRITE_RAM	Activa la escritura secuencial del bloque de 24 bytes en la RAM.

0	1	0	TX_ROM	Transmisión serial de los 4 bytes de cabecera fija (0x00 a 0x03).
0	1	1	TX_RAM	Transmisión serial de los 24 bytes de carga útil del sensor (0x00x17).
1	0	0	TX_STATUS	Transmisión de los bytes de salud interna y telemetría (0x1A a 0x1C).
1	0	1	TX_CHKSUM	Inserción y envío del byte de paridad calculado por hardware (0x1C).
1	1	0	DONE	Finalización del ciclo de trama y liberación de líneas de control.

Tabla 4. Tabla de Transición de Estados de la FSM.

Estado Actual	I2C_RDY	MAX_RAM	MAX_ROM	UB	Siguiente Estado
IDLE	0	X	X	X	IDLE
IDLE	1	X	X	X	WRITE_RAM
WRITE_RAM	X	X	X	X	TX_ROM
TX_ROM	X	X	0	X	TX_ROM
TX_ROM	X	X	1	X	TX_RAM
TX_RAM	X	0	X	X	TX_RAM
TX_RAM	X	1	X	X	TX_STATUS
TX_STATUS	X	X	X	0	TX_STATUS
TX_STATUS	X	X	X	1	TX_CHKSUM
TX_CHKSUM	X	X	X	X	DONE
DONE	X	X	X	X	IDLE

Tabla 5. Tabla de Verdad de Estados y sus Salidas

Estado Actual	Entradas Activas	WR_EN	SEL_MUX	TX_START	INC_PTR (Mealy)	CLR_PTR (Mealy)	LD_ROM (Mealy)
IDLE	I2C_RDY = 0	0	0	0	0	1	0
IDLE	I2C_RDY = 1	0	0	0	0	1	1

WRITE_RAM	Ninguna	1	0	0	0	0	0
TX_ROM	MAX_ROM = 0	0	0	1	1	0	1
TX_ROM	MAX_ROM = 1	0	0	1	1	0	1
TX_RAM	MAX_RAM = 0	0	0	1	1	0	0
TX_RAM	MAX_RAM = 1	0	0	1	1	0	0
TX_STAT_US	UB = 0	0	0	1	1	0	0
TX_STAT_US	UB = 1	0	0	1	1	0	0
TX_CHKSUM	Ninguna	0	1	1	0	0	0
DONE	Ninguna	0	0	0	0	0	0

Implementación a Nivel de Transferencia de Registros (RTL)

Código HDL Estructurado usando una arquitectura modular

Definición de la Interfaz Física

```
module top_telemetry_framer (
    input wire clk_50M,
    input wire rst_n,
    inout wire i2c_sda,
    output wire i2c_scl,
    output wire uart_tx
);
```

En este bloque se definen los enlaces entre el bloque de código y los pines físicos del FPGA y se establecen las entradas dedicadas (clk_50M, rst_n), las salidas para los periféricos (i2c_scl, uart_tx) y los puertos bidireccionales necesarios para topologías de drenador abierto (inout wire i2c_sda).

Declaración de Señales Internas

```
wire clk_100k;
wire clk_uart;
wire data_ready;
wire [191:0] payload_data;
```

```

wire [4:0] mem_ptr;
wire sel_rom, we_ram, load_tx;
wire clear_chk, update_chk, send_chk;
wire [7:0] data_from_mem, checksum_val;
wire tx_busy;
wire [7:0] data_to_uart;

```

Estas señales conforman la lista de conexiones funcional interna. Señales vectoriales como wire [191:0] payload_data o wire [7:0] data_to_uart clasifican como las conexiones por donde fluirá la información útil entre el sensor y la salida. Señales escalares como data_ready, sel_rom, we_ram, y tx_busy representan los cables de bandera (flags) y señales de habilitación que la Máquina de Estados (FSM) utilizará para orquestar los bloques. Finalmente, clk_100k y clk_uart son redes de reloj derivadas que deberán ser enrutadas preferiblemente por la red de reloj global del chip para evitar sesgos (clock skew).

Divisores de reloj

```

clock_divider clk_div_inst (
    .clk_50M(clk_50M),
    .rst_n(rst_n),
    .clk_100k(clk_100k),
    .clk_uart(clk_uart)
);

```

Este bloque es fundamental para adaptar las bases de tiempo a las interfaces periféricas. Toma el reloj global del sistema (clk_50M, ruteado a un pin dedicado de baja capacitancia) y, mediante contadores síncronos, sintetiza dos nuevos dominios de reloj derivados:

- clk_100k: Reloj de 100 kHz destinado a regir la máquina de estados del Master I2C, cumpliendo con el Standard Mode del bus I2C.
- clk_uart: Reloj adaptado al baud rate requerido para la transmisión serie (por ejemplo, 115200 Hz).

Interfaz de Adquisición

```

i2c_master_sensor i2c_master_inst (
    .clk_100k(clk_100k),
    .rst_n(rst_n),
    .scl(i2c_scl),
    .sda(i2c_sda),
    .data_ready(data_ready),
    .payload_data(payload_data)
);

```

Este módulo maneja la capa física y de enlace del protocolo I2C bidireccional (Open-Drain). Actúa como maestro en el bus, generando la señal de sincronismo scl y gestionando las transiciones de la línea sda para leer la telemetría del sensor externo. Una vez que la transacción I2C finaliza, el módulo empaqueta los bits

capturados en un bus paralelo masivo de 192 bits (payload_data) y afirma una bandera de un solo ciclo (data_ready) avisando a la FSM gobernadora que el dato es válido y seguro para ser muestreado.

Sistema de Memoria y Datapath

```
memory_and_framer mem_framer_inst (  
    .clk(clk_50M),  
    .rst_n(rst_n),  
    .addr(mem_ptr),  
    .sel_rom(sel_rom),  
    .we_ram(we_ram),  
    .payload_data(payload_data),  
    .clear_checksum(clear_chk),  
    .update_checksum(update_chk),  
    .data_out(data_from_mem),  
    .checksum(checksum_val)  
);
```

Este bloque encapsula la ruta de datos (Datapath) pura. Abstrae la instanciación de memorias y de lógica aritmética:

- Manejo de Memoria: Recibe un puntero de dirección (addr) de 5 bits y señales de control (sel_rom, we_ram) para multiplexar lógicamente entre una ROM (donde seguramente residen las palabras de sincronismo o cabeceras de la trama) y una RAM (o banco de registros) que almacena temporalmente el arreglo payload_data.
- Calculo de Checksum: Incluye hardware aritmético para acumular y calcular el checksum byte a byte en tiempo real, gobernado por las señales clear_checksum y update_checksum.

En la salida se presenta el byte de memoria solicitado (data_from_mem) y el estado actual del cálculo de paridad/suma (checksum_val).

Lógica de Selección a Nivel Top

```
assign data_to_uart = send_chk ? checksum_val : data_from_mem;
```

Es una asignación continua que infiere un multiplexor combinacional (MUX) 2-a-1. La FSM gobernadora usa la señal send_chk como selector: si está inactiva, la UART se alimenta de la secuencia de bytes proveniente de la memoria (cabeceras + datos); si se activa, la UART recibe el byte de validación de trama (checksum).

Unidad de Control Resiliente (Framer FSM TMR)

```
framer_fsm_tmr fsm_tmr_inst (  
    .clk_50M(clk_50M),  
    .rst_n(rst_n),
```



```

        .data_ready_100k(data_ready),

        .tx_busy(tx_busy),

        .we_ram(we_ram),

        .sel_rom(sel_rom),

        .load_tx(load_tx),

        .ptr(mem_ptr),

        .clear_checksum(clear_chk),

        .update_checksum(update_chk),

        .send_checksum(send_chk)

    );

```

Esta es la ruta de control (Control Path) y el orquestador del sistema, este bloque implementa tres máquinas de estado finitas (FSM) que operan en paralelo y someten su estado lógico a una votación mayoritaria (Voter) para tolerar y mitigar caídas y errores transitorios (Single Event Upsets - SEUs). Su lógica evalúa entradas de estado (`data_ready`, `tx_busy`) y transiciona en cada ciclo de `clk_50M` para disparar una orquestación precisa: limpia el checksum, guarda el payload, barre el puntero (`mem_ptr`) leyendo la ROM y RAM, actualiza el checksum, e instruye iterativamente a la UART con `load_tx` para transmitir byte tras byte hasta finalizar el ensamblado de la trama.

Transmisor Serie Asíncrono (UART TX module)

```

uart_tx_module uart_tx_inst (

    .clk_uart(clk_uart),

    .rst_n(rst_n),

    .tx_start(load_tx),

    .data_in(data_to_uart),

    .tx_pin(uart_tx),

    .tx_busy(tx_busy)

);

```

Es el periférico de salida, un serializador implementado mediante un registro de desplazamiento (Shift Register). Al recibir el pulso de confirmación `tx_start` (vinculado a `load_tx`), toma el byte paralelo `data_in` (proveniente del MUX) e inicia el enmarcado UART (Start bit, Data bits, Stop bit) y levanta la bandera `tx_busy` mientras se encuentra desplazando los bits físicamente por el pin de silicio `uart_tx` a la frecuencia impuesta por `clk_uart`. Esto es fundamental para hacer un handshake con la FSM gobernadora (la FSM detiene su iteración y espera a que `tx_busy` baje a '0' antes de enviarle el siguiente byte).

Plan de Verificación y Testbench:

El proceso de validación del Ensamblador de Tramas de Telemetría se fundamenta en una estrategia de simulación técnica mediante un banco de pruebas lógico síncrono, denominado *tb_telemetry_framer*. Este enfoque busca aislar y analizar exhaustivamente la Unidad Bajo Prueba (UUT: *top_telemetry_framer*) bajo condiciones operativas críticas

dentro de un entorno virtual controlado. Para garantizar la correcta verificación a Nivel de Transferencia de Registros (RTL) de la FSM que coordina el sistema, es esencial que el Testbench (TB) instancie el Dispositivo Bajo Prueba (DUT) y lo someta a una secuencia de estímulos técnicos rigurosamente supervisados.

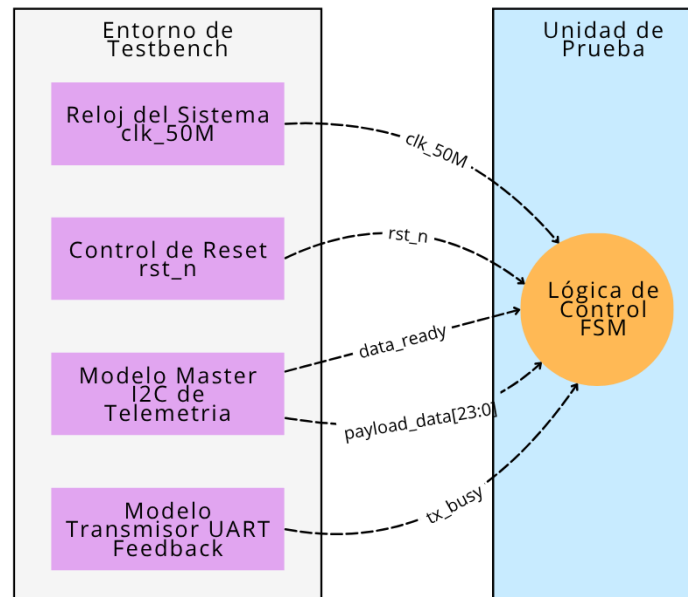


Figura 4. Diagrama de estímulos de prueba

Tabla 6. Matriz de Casos de Prueba

Casos de Prueba	Descripción y Estimulos inyectados	Comportamiento y Salidas Esperadas
Caso 1: Flujo Nominal (Ensamblaje Exitoso)	Se liberará el control de reset en 1 y se inyectará un pulso de data_ready junto con un payload_data válido. Se manipula tx_nusy emulando los retardo mninales de un UART a 115200 bps.	La FSM debe abandonar IDLE, almacenar los datos, barrer el puntero mem_ptr y emitir una señal load_tx byte a byte. El bloque de control debe esperar a que tx_busy sea 0 antes de cada transmisión de byte. Finalmente el estado de send_cheksum debe asentarse correctamente.
Caso 2: Comportamiento ante un reset a medio flujo	Se iniciará el ensamblaje como en el Caso 1, pero justo cuando el FSM está en estado de transmisión del segundo byte del payload, se inserta un control de reset en 0.	La FSM debe abortar inmediatamente la transacción en el flanco de bajada de rst_n, es así que el puntero de memoria debe volver a 0, las señales de control load_Tx y wer_ram deben bajar a 0, volviendo al estado IDLE.
Caso 3 : Interrupción de flujo de datos (Pérdida de señal)	Se liberará el reset y el reloj para que corran libremente, pero se mantendrá data_ready en 0 de forma indefinida o por tiempos muy prolongados.	La FSM debe mantenerse estable y bloqueada en el estado IDLE. No debe existir ninguna transición, ni se deberian de activar las señales de escritura en memoria o inicialización de UART.
Caso 4: Transiciones a estados de error (tx_busy	Se inyectará data_ready y el sistema iniciará el ensamblaje, pero se forzará tx_busy en 1 sin liberarlo, de esta forma emularemos el cuelgue del periférico UART.	La FSM debe transicionar a su estado de espera y permanecer ahí. Se verificará que no ocurra un desbordamiento del puntero de memoria y que no se

timeout/stall)		sobreescriba la RAM.
----------------	--	----------------------

Con el Caso 1 se busca garantizar que la FSM abandone el estado de reposo y recorra todos los estados operativos, validando simultáneamente las transiciones síncronas de avance necesarias para iterar, encolar y transmitir la trama byte a byte; con el Caso 2, se valida que el sistema sea capaz de abortar a medio flujo de forma controlada, verificando los arcos de escape hacia IDLE ante un reinicio para evitar que el hardware quede atrapado en un estado inconsistente; con el Caso 3, se garantiza la cobertura estática del estado IDLE, asegurando que el sistema pueda permanecer en bajo consumo y sin alterar la memoria cuando no hay estímulos; y en el Caso 4, se asegura que la FSM alcance estados límite que no se dan en un flujo ideal, evaluando correctamente las transiciones de retención (auto-bucles) ante esperas prolongadas para garantizar que no existan desbordamientos de memoria ni corrupción en el cálculo del checksum.

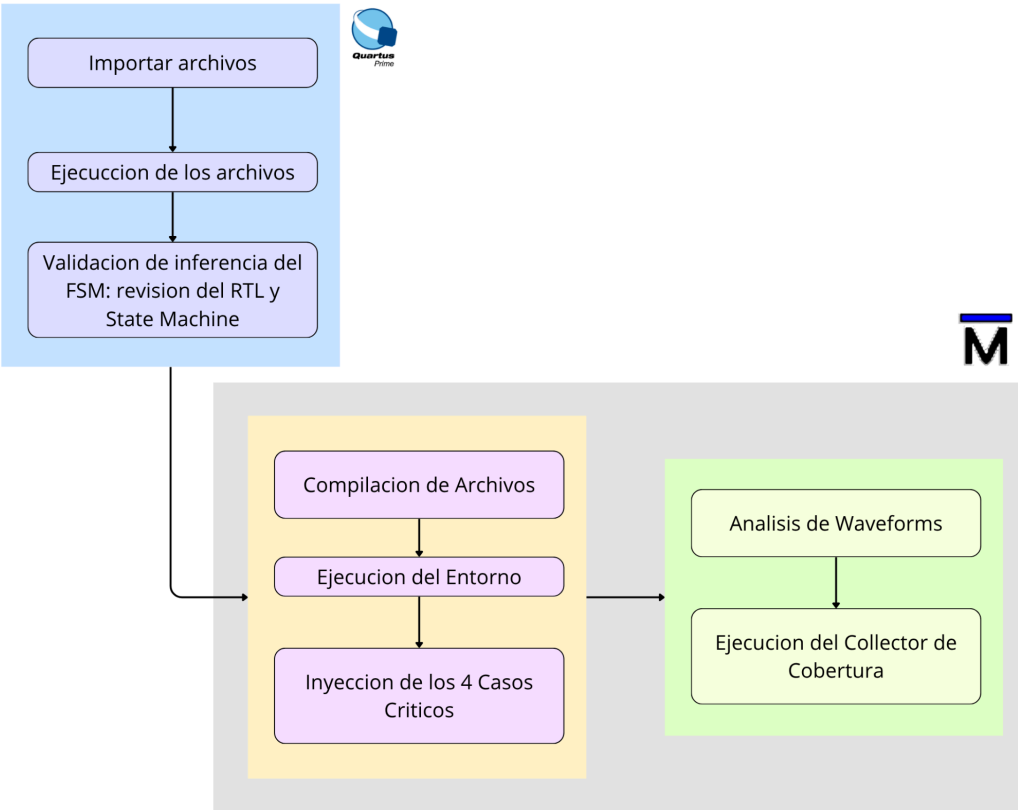


Figura 5. Flujo de Simulación en Quartus y ModelSim

Tabla 6. Estructura Secuencial y Distribución de Campos de la Trama de Telemetría de 13 Bytes

Índice del Byte	Segmento de Trama	Valor Hexadecimal Esperado	Origen Físico	Función del Sistema
0	Frame Header	0xAA	ROM Estática	Sincronización del Receptor (SYNC1)

1	Frame Header	0x55	ROM Estática	Sincronización del Receptor (SYNC2)
2	Frame Header	0x01	ROM Estática	Identificador de Identidad (DEVICE_ID)
3	Frame Header	0x18	ROM Estática	Longitud Útil del Marco (PAYLOAD_LEN)
4-17	Payload RAM	[Varía]	Sensor I2C (MPU6050)	Carga útil: Acelerómetro, Giroscopio y Temperatura. (14 bytes)
18-25	Payload RAM	[Vara]	Sensor I2C (BME280)	Carga útil: Presión, Temperatura y Humedad. (8 bytes)
26-27	Payload RAM	[Vara]	Sensor I2C (BH1750)	Carga útil: Sensor de luz ambiental / Lux (2 bytes).
28	Payload RAM	[Calculado XOR]	Multiplexor (Registro XOR)	Bit de redundancia cíclica dinámico calculado para validación de tramas.

Resultados de Simulación y Análisis de Desempeño

Validación de Inferencia del FSM

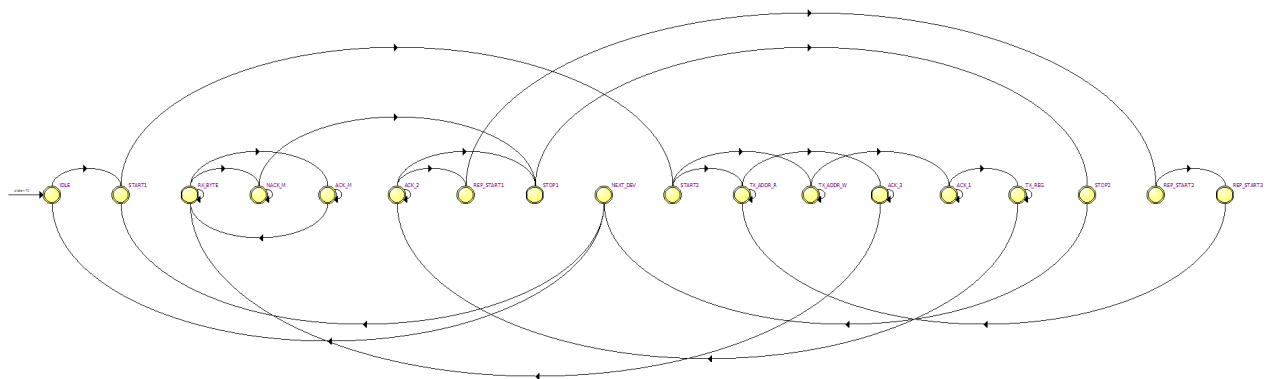


Figura 6. FSM inferido por Quartus

Source State	Destination State	Condition
1 ACK_1	ACK_1	(!phase)
2 ACK_1	TX_REG	(phase)
3 ACK_2	STOP1	(phase) (device_idx[0]) (device_idx[1]) (b[1750_init_done) + (phase) (device_idx[0]) (device_idx[1])
4 ACK_2	REP_START1	(phase) (device_idx[0]) (device_idx[1]) + (phase) (device_idx[0]) (device_idx[1]) (b[1750_init_done) + (phase) (device_idx[0]) (device_idx[1])
5 ACK_2	ACK_2	(!phase)
6 ACK_3	ACK_3	(!phase)
7 ACK_3	RX_BYTE	(phase)
8 ACK_M	ACK_M	(!phase)
9 ACK_M	RX_BYTE	(phase)
10 IDLE	START1	
11 NACK_M	STOP1	(phase)
12 NACK_M	NACK_M	(!phase)
13 NEXT_DEV	START1	(device_idx[0]) (device_idx[1]) + (device_idx[0])
14 NEXT_DEV	IDLE	(device_idx[0]) (device_idx[1])
15 REP_START1	REP_START2	
16 REP_START2	REP_START3	
17 REP_START3	TX_ADDR_R	
18 RX_BYTE	NACK_M	(phase) (bytes_to_read[0]) (bytes_to_read[1]) (bytes_to_read[2]) (bytes_to_read[3]) (bytes_to_read[4]) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2])
19 RX_BYTE	ACK_M	(phase) (bytes_to_read[0]) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2]) + (phase) (bytes_to_read[0]) (bytes_to_read[1]) (bytes_to_read[2]) (bytes_to_read[3]) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2]) + (phase) (bytes_to_read[0]) (bytes_to_read[1]) (bytes_to_read[2]) (bytes_to_read[3]) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2])
20 RX_BYTE	RX_BYTE	(phase) (bytes_to_read[0]) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2]) + (phase) (bytes_to_read[0]) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2]) + (phase) (bytes_to_read[0]) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2]) + (phase) (bytes_to_read[0]) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2])
21 START1	START2	
22 START2	TX_ADDR_W	(device_idx[0]) (device_idx[1]) + (device_idx[0]) (device_idx[1]) (b[1750_init_done) + (device_idx[0]) (device_idx[1])
23 START2	TX_ADDR_R	(device_idx[0]) (device_idx[1]) (b[1750_init_done) + (device_idx[0]) (device_idx[1])
24 STOP1	STOP2	
25 STOP2	NEXT_DEV	
26 TX_ADDR_R	TX_ADDR_R	(phase) + (phase) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2]) + (phase) (bit_cnt[0]) (bit_cnt[1]) + (phase) (bit_cnt[0])
27 TX_ADDR_R	ACK_3	(phase) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2])
28 TX_ADDR_W	TX_ADDR_W	(phase) + (phase) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2]) + (phase) (bit_cnt[0]) (bit_cnt[1]) + (phase) (bit_cnt[0])
29 TX_ADDR_W	ACK_1	(phase) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2])
30 TX_REG	TX_REG	(phase) + (phase) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2]) + (phase) (bit_cnt[0]) (bit_cnt[1]) + (phase) (bit_cnt[0])
31 TX_REG	ACK_2	(phase) (bit_cnt[0]) (bit_cnt[1]) (bit_cnt[2])

Figura 7. Tabla FSM inferida por Quartus de los estados iniciales y los estados de destino

Name	NEXT_DEV	STOP2	STOP1	NACK_M	ACK_M	RX_BYTE	ACK_3	TX_ADDR_R	REP_START3	REP_START2	REP_START1	ACK_2	TX_REG	ACK_1	TX_ADDR_W	START2	START1	IDLE
1 IDLE	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2 START1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
3 START2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
4 TX_ADDR_W	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1
5 ACK_1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1
6 TX_REG	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1
7 ACK_2	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1
8 REP_START1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1
9 REP_START2	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1
10 REP_START3	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1
11 TX_ADDR_R	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1
12 ACK_3	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1
13 RX_BYTE	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1
14 ACK_M	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1
15 NACK_M	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1
16 STOP1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
17 STOP2	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
18 NEXT_DEV	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Figura 8. Tabla FSM inferida por Quartus codificada

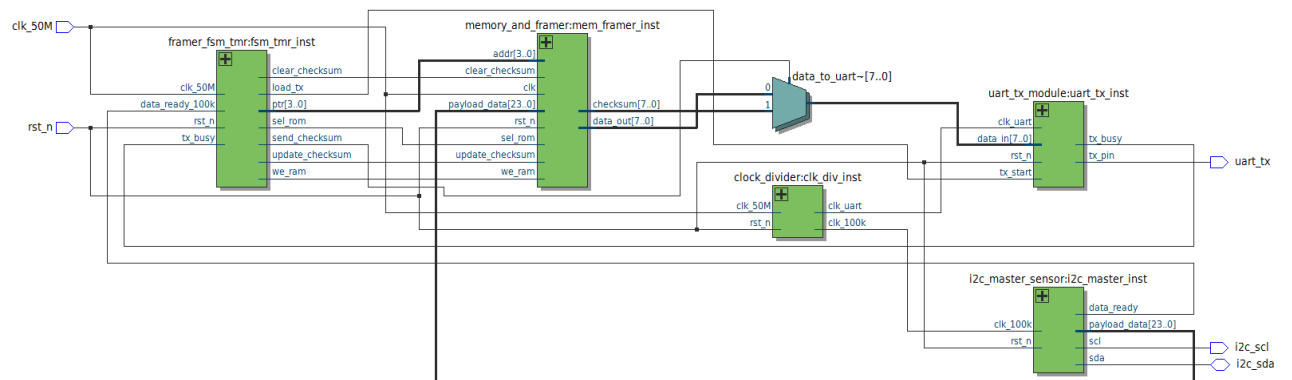
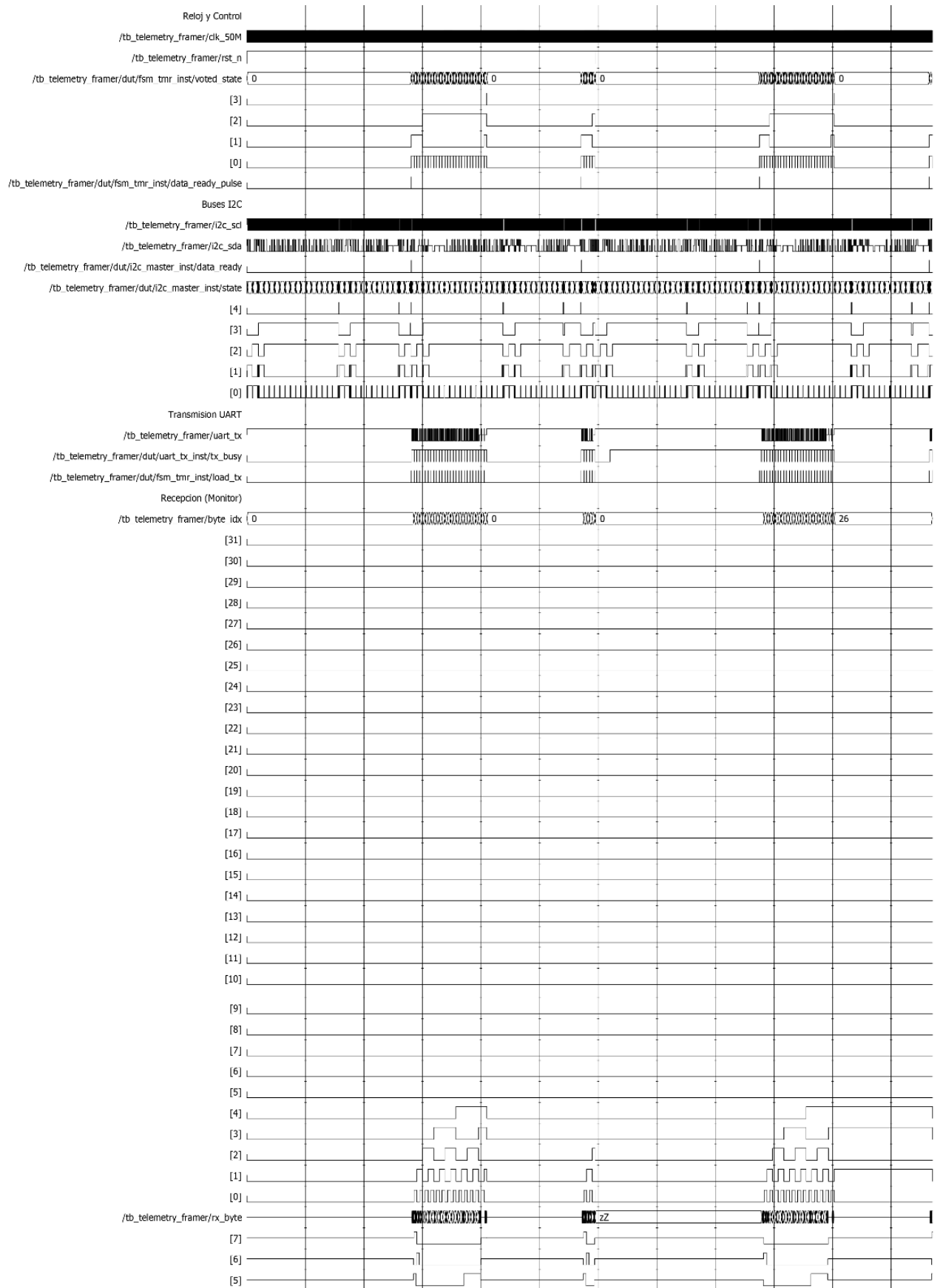


Figura 9. Diagrama RTL inferida por Quartus

Análisis de Formas de Onda por Casos (Waveforms)



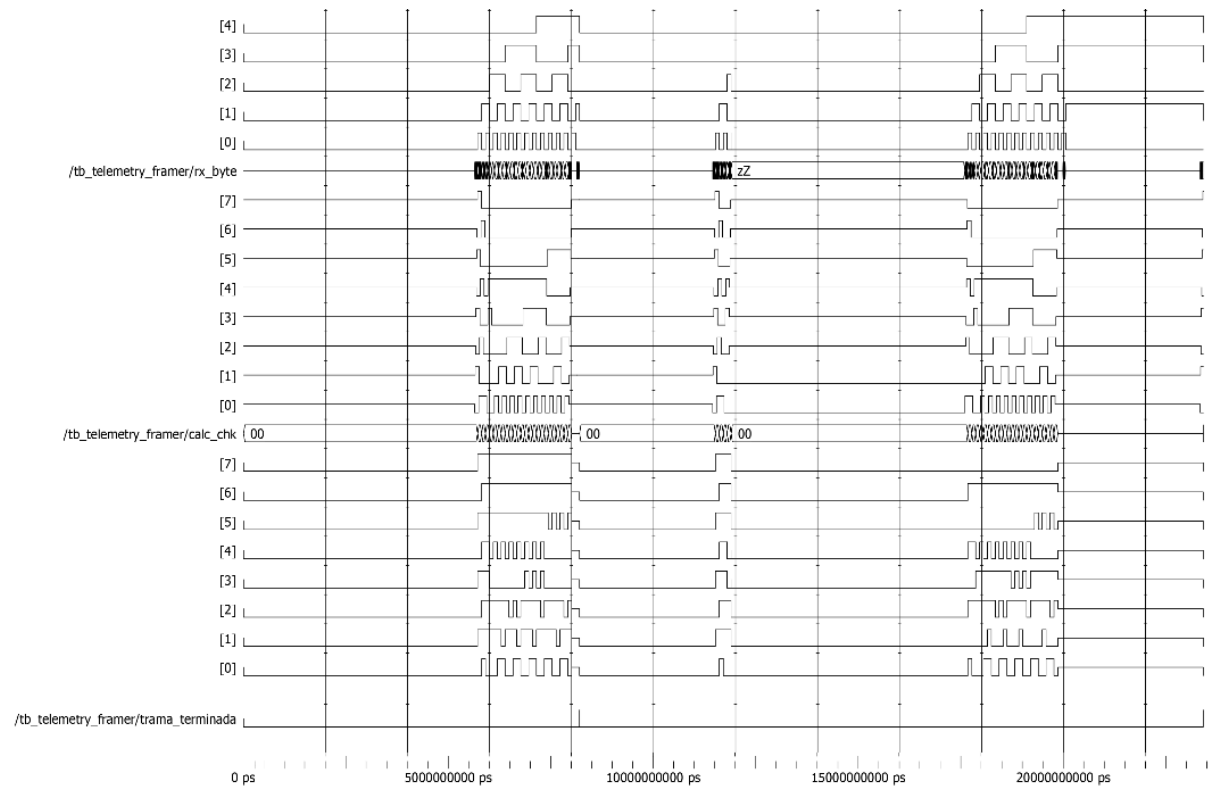
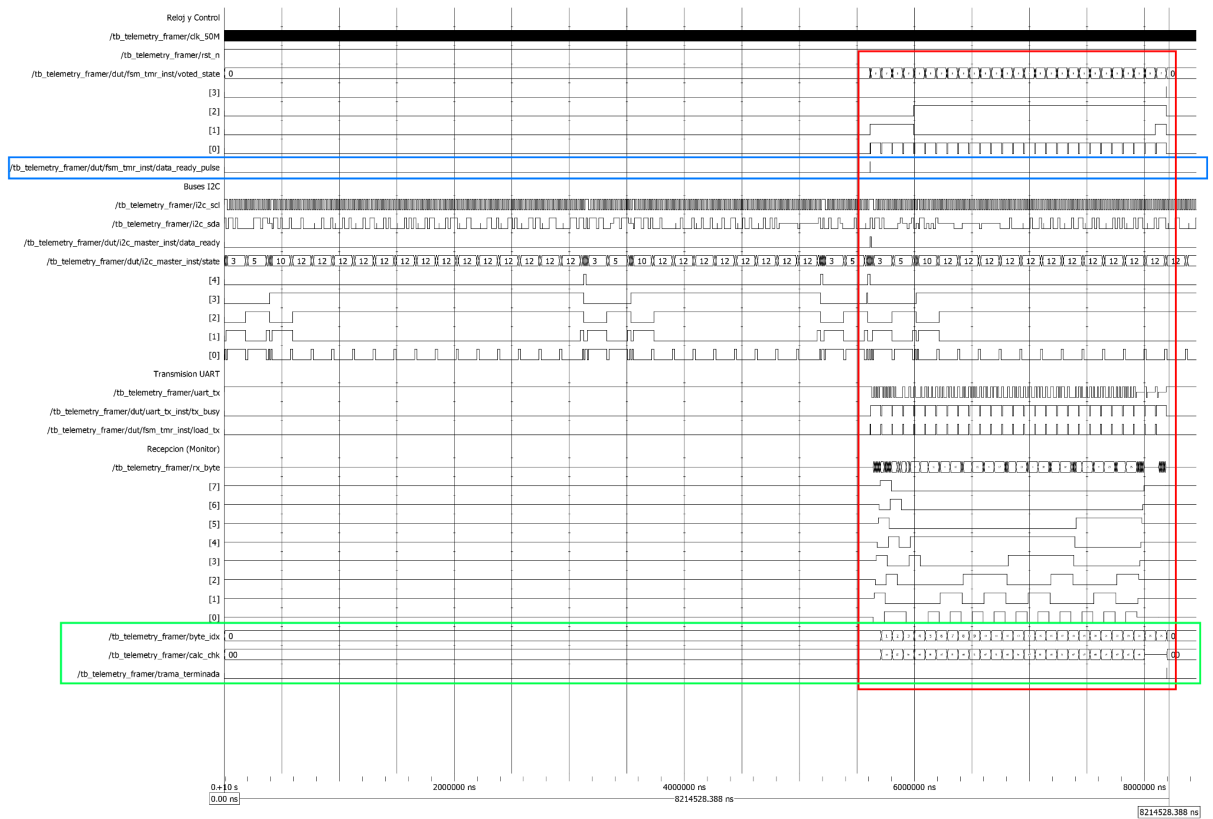


Figura 11. Waveform del test del sistema completo



Entity: tb_telemetry_framer Architecture: Date: Sun Jul 05 19:20:25 EDT 2026 Row: 1 Page: 1

Figura 12. Waveform del Caso 1

El ciclo de funcionamiento óptimo del sistema se evidencia en la primera captura presentada.

- Área Azul: Documenta el momento en que culmina la lectura del bus I2C y se activa la bandera correspondiente, emitiendo un pulso nítido en data_ready_pulse. Este evento funciona como el disparador de sincronía ideal que permite al voted_state de la FSM transicionar fuera del estado de reposo o IDLE.
- Área Roja: Ilustra la etapa de transmisión de alta densidad. En este punto, la FSM gestiona con precisión el intercambio de señales (handshake) con la UART, alternando entre fases de alistamiento y latencia, tales como los estados TX_ROM_WAIT y TX_RAM_WAIT. Es aquí donde la señal uart_tx ejecuta la serialización física de la información.
- Área Verde: El éxito de la secuencia operativa es ratificado por el monitor del Testbench. Se aprecia el incremento secuencial de la variable byte_idx a medida que se procesan los bytes; al procesar el Checksum final, el monitor confirma la paridad, reinicia calc_chk a cero y activa el pulso trama_terminada, lo que certifica la integridad matemática del paquete completo.

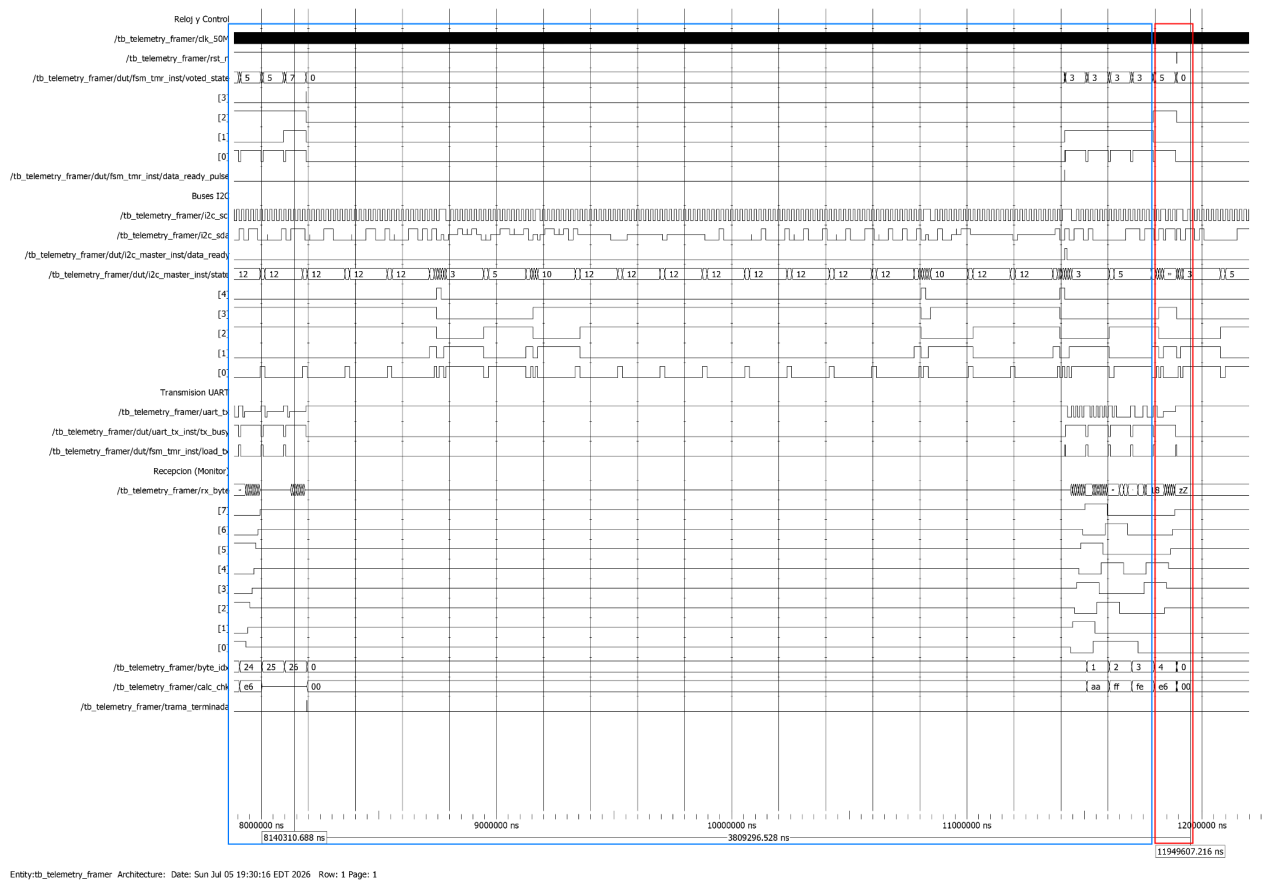
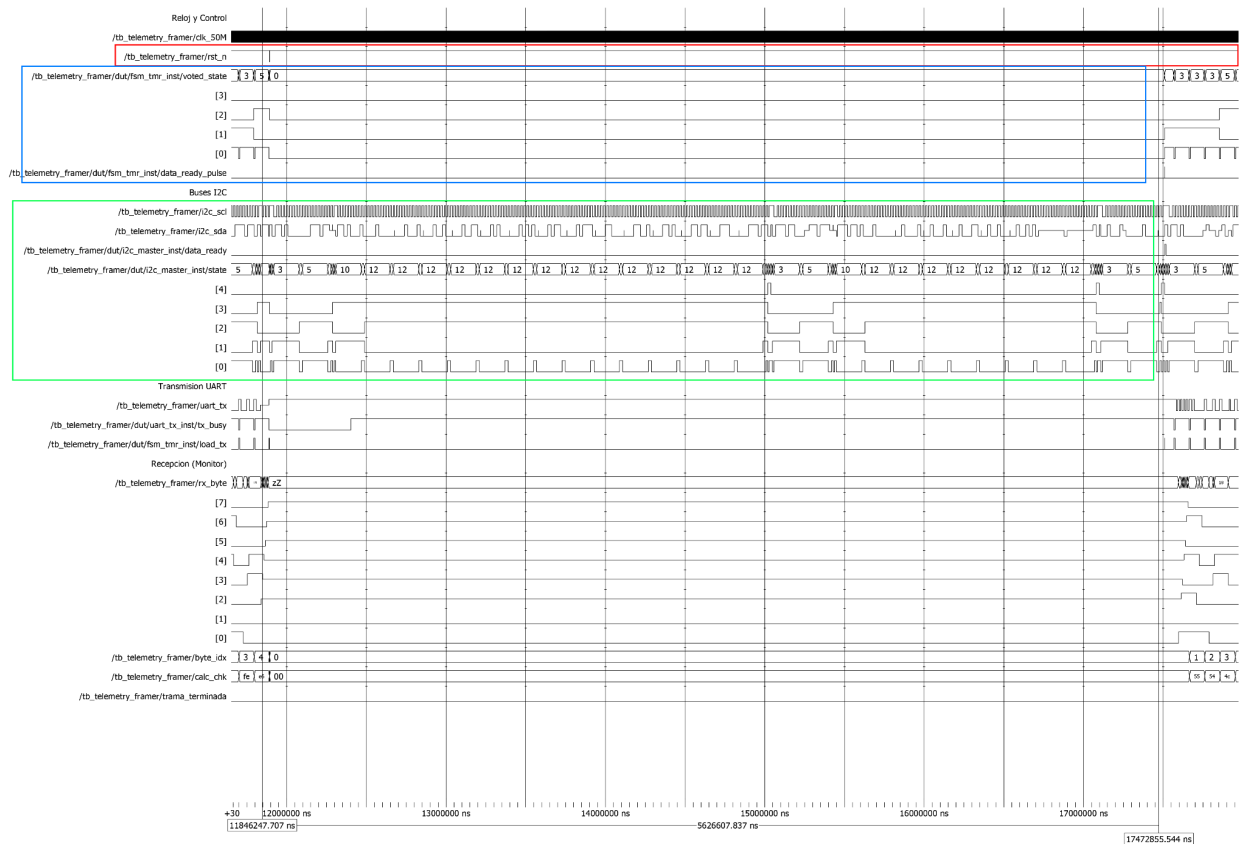


Figura 13. Waveform del Caso 2

El propósito de esta imagen es analizar la capacidad de recuperación del sistema frente a reinicios asíncronos inesperados.

- Franja Azul: Establece el periodo de prueba en el que el sistema funcionaba de manera regular y el monitor UART (byte_idx) ya había registrado los bytes iniciales de la estructura de datos (índices 3, 4, etc.).

- **Franja Roja:** Representa el instante preciso de la inducción del error, donde se aprecia el descenso súbito de la señal de reinicio (rst_n a 0). De forma simultánea, el registro de votación de la FSM (voted_state) interrumpe el proceso en memoria y regresa de manera protegida al estado inicial (IDLE). Al mismo tiempo, la salida uart_tx finaliza su actividad de forma nítida. Este comportamiento valida que la arquitectura evita bloqueos en estados transitorios erróneos y asegura que los índices vuelvan a su origen, permitiendo una reanudación fiable de las operaciones.



Entity: tb_telemetry_framer Architecture: Date: Sun Jul 05 19:29:34 EDT 2026 Row: 1 Page: 1

Figura 14. Waveform del Caso 3

Mediante esta prueba se valida la estabilidad de la máquina de estados frente a la inexistencia de señales externas, como podría ocurrir ante una desconexión del bus I2C o averías en los sensores.

- **Segmento Rojo:** Se verifica que el sistema se encuentra fuera del estado de reinicio (rst_n en nivel alto), permitiendo la libre operación de la FSM.
- **Segmento Verde:** Se evidencia actividad persistente en el bus I2C (i2c_scl e i2c_sda), lo que simula intentos de lectura del maestro. No obstante, se ha condicionado el sistema para que la señal data_ready no se active, emulando una falla crítica en la captura del payload.
- **Segmento Azul:** Refleja la respuesta prevista; el pulso data_ready_pulse se mantiene inactivo. Consecuentemente, el estado de la FSM (voted_state) permanece fijo en el estado 0 (IDLE). Esta ausencia de transiciones o desbordamientos asegura que no se transmitirá información errónea por la UART ni se producirán sobreescrituras en la RAM si la integridad de la carga útil no está plenamente asegurada.

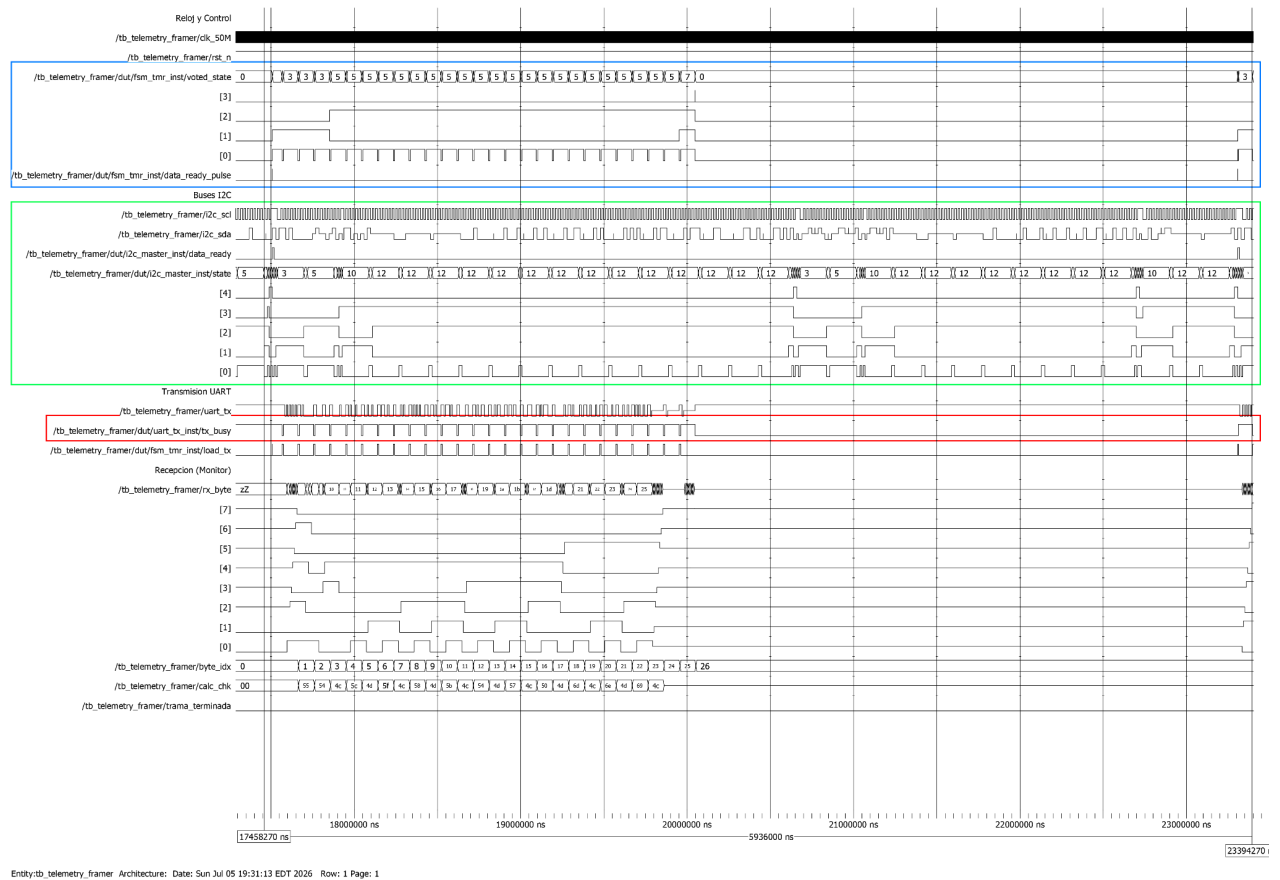


Figura 15. Waveform del Caso 4

Mediante esta captura se valida la capacidad de retención (auto-bucles) del sistema frente a un bloqueo en el periférico de salida (UART).

- Segmento Azul: Se aprecia que, tras el pulso data_ready, el sistema inicia su operación con normalidad; no obstante, al alcanzar el estado 5 (TX_RAM_WAIT), la señal voted_state se estabiliza en una línea recta, indicando una pausa indefinida.
- Segmento Rojo: Identifica la causa de la detención. La señal tx_busy, emitida por el módulo UART, se ha forzado a un nivel alto constante (1), simulando una interrupción en el hardware de transmisión.

La relevancia de este ensayo radica en confirmar la robustez de la FSM para pausar su ejecución. En lugar de proseguir con la inyección de datos ignorando el estado de la UART —lo que derivaría en la corrupción de la trama y la memoria—, el estado 5 preserva íntegro el valor del puntero ptr_reg. Al liberarse la señal tx_busy hacia el final del registro gráfico, la FSM reanuda su ciclo operativo de forma fluida y transparente, lo que ratifica la efectividad del diseño del Handshake.

Definición Teórica de Métricas

- *Consumo de Recursos Lógicos:*

Esta métrica evalúa el nivel de ocupación en el chip, lo que impacta directamente en la potencia y la sensibilidad a errores. Para poder realizar la cuantificación usaremos el siguiente modelo matemático:

$$Utilización (\%) = \left(\frac{Recursos\ Instanciados}{Recursos\ Totales\ del\ Dispositivo} \right) \times 100$$

Tomando en cuenta que el dispositivo que usaremos (Cyclone IV) cuenta con 6272 Elementos Lógicos (LEs), donde cada LE contiene una tabla de búsqueda (LUT) de 4 entradas y un registro. Cuenta con 30 bloques de memoria M9K (cada uno de 9 Kbits) y 15 multiplicadores embebidos de 18×18 bits..

- *Rendimiento de Temporizador y Throughput:*

Garantiza el determinismo y la velocidad de procesamiento del hardware interno.

Frecuencia Máxima:

$$f_{max} = \frac{1}{T_{critical-path}}$$

Setup Slack:

$$Slack = Data\ Required\ Time - Data\ Arrival\ Time.$$

Un $Slack \geq 0$, garantiza que no hay violaciones de temporización.

Throughput:

$$Throughput\ (bps) = Ancho\ del\ Bus\ (bits) \times f_{reloj}$$

Al usar el grado de Velocidad C8, el rendimiento máximo teórico disminuye respecto a chips más rápidos. La red de reloj del dispositivo soporta un máximo de 402 MHz. Los bloques de memoria M9K operan a un máximo de 238 MHz, y los multiplicadores de 18 × 18 bits alcanzan 200 MHz en este grado de velocidad. Para tu lógica de enrutamiento y FSM, un reloj base del sistema de 50 MHz garantiza un Slack positivo extraordinariamente amplio y seguro. En este caso, a 50 MHz, el ancho de banda interno sobra por completo para interconectar I2C (100 kbps - 400 kbps) y UART (115200 bps o 1-2 Mbps).

- *Latencia Determinista End-to-End:*

Evalúa el determinismo exacto de los paquetes fluyendo por el hardware.

$$Latencia_{E2E} = \sum (Ciclos\ de\ reloj\ por\ etapa) \times T_{reloj}$$

Como el hardware no posee sistema operativo (no hay interrupciones ni jitter de software), el paso por la FSM dependerá únicamente del diseño síncrono.

Se espera que como se está Operando a un reloj de placa estándar de 50 MHz ($T_{reloj} = 20\ ns$), el cálculo intermedio del checksum y el enrutamiento de la cola desde la entrada I2C hasta la salida UART requerirá pocos ciclos de reloj (típicamente de 2 a 5). Esto resultará en una latencia determinista estricta en el rango de 40 a 100 nanosegundos, en contraste con los microsegundos/milisegundos que tardaría un microcontrolador convencional.

- *Consumo de Potencia y Eficiencia Energética:*

La potencia disipada afecta la viabilidad térmica del ensamble.

$$P_{total} = P_{estática} + P_{dinámica}$$

Donde $P_{dinámica} \approx C \cdot V^2 \cdot f$

$P_{estática}$: Al ser el EP4CE6 el chip con menor densidad de la familia, la corriente de fuga térmica (y por tanto la potencia en reposo) es intrínsecamente la más baja de todo el catálogo Cyclone IV, requiriendo muy poca disipación pasiva

$P_{dinámica}$: Como sólo se conmutará alrededor de una cuarta parte (~23%) de un chip que ya de por sí es muy pequeño, operando a frecuencias conservadoras (50 MHz), el consumo dinámico será mínimo.

- *Cobertura de Fallos y Sensibilización a Errores (SEUs y SETs):*

Esta métrica predice matemáticamente la probabilidad de que una partícula o ruido cause un fallo en tu FSM o registros.

Tasa de Fallos del Circuito:

$$\lambda_{circuito} = \lambda_{dispositivo} \times U \times AVF$$

Donde U es la utilización del chip y AVF es el Factor de Vulnerabilidad Arquitectónica.

Probabilidad de detección concurrente: $P_d = q_{CED}^2$

Donde q_{CED} es la probabilidad de que la circuitería detectora no falle.

Referencias:

- [1] DYC Electronic, "FPGA vs microcontroller understanding the key differences and use cases," PCBA Manufacture in China, May 22, 2026. [Online]. Available: <https://www.dyc-electronic.com/pt/fpga-vs-microcontroller/>. [Accessed: Jun. 6, 2026].
- [2] NI, "More throughput vs. less latency: Understand the difference," Nov. 1, 2013. [Online]. Available: <https://www.ni.com/en/shop/data-acquisition-and-control/add-ons-for-data-acquisition-and-control/what-is-a-bview-real-time-module/make-it-faster--more-throughput-or-less-latency-.html>. [Accessed: Jun. 6, 2026].
- [3] Thai, "Applications for FPGAs on nanosatellites," M.S. thesis, Graduate Program in Earth and Space Science, York University, Toronto, ON, Canada, Apr. 2014.
- [4] R. Tedeschi et al., "Temporal lockstep: Low-cost resilient design for microcontroller-class RISC-V processors," IEEE Access, vol. 14, pp. 1-1, Mar. 2026. doi: 10.1109/ACCESS.2026.3676682.
- [5] M. Duni, "On the development of radiation-hardened high performance computing nodes for satellite systems," Master's thesis, Master's Degree in Aerospace Engineering — Aeromechanics and Systems, Politecnico di Torino, Turin, Italy, Dec. 2025.
- [6] M. Kubica and R. Czerwinski, "Error mitigation methods for FSM using triple modular redundancy," *Appl. Sci.*, vol. 15, no. 12, p. 6726, 2025. doi: 10.3390/app15126726
- [7] D. Agiakatsikas, "High-level synthesis of triple modular redundant FPGA circuits with energy efficient error recovery mechanisms," Ph.D. dissertation, School of Computer Science and Engineering, Faculty of Engineering, The University of New South Wales, Sydney, NSW, Australia, June 2019.
- [8] LCSC Electronics, "Intel (Altera) EP4CE6E22C8N - Programmable Logic Device (CPLDs/FPGAs)," LCSC, 2026. [En línea]. Disponible: https://www.lcsc.com/product-detail/Programmable-Logic-Device--CPLDs-FPGAs-_Intel-Altera-EP4CE6E22C8N_C36238.html. [Accedido: 14-jun.-2026].